

CS464

Data and Network Security

DES Example: Full Walkthrough



Hafiz Syed Ahmed Qasim



ahmed.qasim@giki.edu.pk

Introduction to DES

What is DES?

The Data Encryption Standard (DES) is a symmetric-key algorithm for the encryption of digital data.

DES was developed in the 1970s and published by the National Institute of Standards and Technology (NIST).

Key Characteristics

- DES uses a **56-bit key** (with 8 parity bits making it 64 bits).
- It operates on **64-bit blocks** of plaintext.
- The algorithm consists of **16 rounds** of the Feistel network structure.

Significance

DES was the first standard cryptographic algorithm widely adopted and studied.

It formed the foundation for understanding modern block cipher design and cryptanalysis.

Official Standard

DES is specified in **FIPS PUB 46-3** by NIST.

This document provides the complete specification including all substitution and permutation tables.

Chapter 1

Encryption Process

Data Encryption Standard (DES) - Complete Walkthrough



DES Encryption



Process Overview

- Initial Permutation
- 16 Round Operations
- Final Permutation

Setup: Plaintext and Key



Plaintext

Hexadecimal Value:

0123456789ABCDEF

Binary Representation:

```
0000 0001 0010 0011 0100 0101
0110 0111 1000 1001 1010 1011
1100 1101 1110 1111
```



Key

Hexadecimal Value:

133457799BBCDFF1

Binary Representation:

```
0001 0011 0011 0100 0101 0111
0111 1001 1001 1011 1011 1100
1101 1111 1111 0001
```

Step 1: Initial Permutation (IP)

↔ Process

The 64-bit plaintext is rearranged according to the **fixed Initial Permutation (IP) table**.

This step reorders the bits to create a more uniform distribution before the Feistel rounds.

➡ IP Output

CC00CCFF F0AAF0AA

The 64-bit block is split into two 32-bit halves: Left (L_0) and Right (R_0).

↗ Split Result

L_0 (Left Half):

CC00CCFF

Binary: 1100 1100 0000 0000 1100 1100 1111 1111

R_0 (Right Half):

F0AAF0AA

Binary: 1111 0000 1010 1010 1111 0000 1010 1010

Purpose of IP



The Initial Permutation provides diffusion and makes subsequent operations more effective by distributing the plaintext bits across the entire block.

Initial Permutation (IP)

IP Table Overview

- The Initial Permutation (IP) is a **64-bit table** that rearranges the bits of the input plaintext.
- Each entry in the table specifies which input bit position is moved to that output position.
- This permutation is the **first step** in both encryption and decryption processes.

Purpose

The IP table provides **diffusion** by spreading the plaintext bits across the entire block before the Feistel rounds begin.

IP

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Step 2: Key Generation (Round 1 Subkey)

⚙️ Generation Process

- The 64-bit key drops its 8 parity bits.
- Permuted via **PC-1** table.
- Divided into two 28-bit halves: C_0 and D_0 .

⌞⌟ Initial Halves

C_0 (Binary):

```
1111 0000 1100 1100 1010 1010 1111
```

D_0 (Binary):

```
0101 0101 0110 0110 0111 1000 1111
```

➔ Round 1 Shift

Both halves are **left-shifted by 1 position** to produce C_1 and D_1 .

This rotation creates the key schedule for each round.

↔ Shifted Halves

C_1 :

```
1110 0001 1001 1001 0101  
0101 1111
```

D_1 :

```
1010 1010 1100 1100 1111  
0001 1110
```

🔑 Round 1 Subkey (K_1)

Combined and passed through **PC-2** table:

```
0001 1011 0000 0010 1110 1111 1111 1100  
0111 0000 0111 0010
```

Permuted Choice 1 (PC-1)

PC-1

Left

Right

57	49	41	33	25	17	9	63	55	47	39	31	23	15
1	58	50	42	34	26	18	7	62	54	46	38	30	22
10	2	59	51	43	35	27	14	6	61	53	45	37	29
19	11	3	60	52	44	36	21	13	5	28	20	12	4

Permuted Choice 2 (PC-2)

PC-2

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

Step 3: Round 1 Detail (A-B)



Expansion (E-Box)

The right half, R_0 (32 bits), is **expanded to 48 bits** by duplicating 16 specific bits.



This expansion allows each bit of R_0 to affect two S-Boxes.

$E(R_0)$ Output (Binary):

```
0111 1010 0001 0101 0101 0101 0111 1010
      0001 0101 0101 0111
```



Key Mixing (XOR)

The expanded R_0 is **XORed with the Round 1 subkey (K_1)**.



This operation introduces the key material into the round function.

$E(R_0)$ XOR K_1 (Binary):

```
0110 0001 0001 0111 1011 1010 1000 0110
      0110 0101 0010 0101
```

Expansion Function (E)

E-Table Overview

- The Expansion Function (E) expands the **32-bit right half** to **48 bits** by duplicating specific bits.
- Each output bit position specifies which input bit to copy, allowing some input bits to appear **twice** in the output.
- This expansion enables each bit of R_n to affect **two S-Boxes** during the substitution phase.

Purpose

The expansion increases the **avalanche effect** by ensuring that small changes in the input propagate through multiple S-Boxes.

E

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Step 3: Round 1 Detail (C-D)

Substitution (S-Boxes)

The 48-bit result is split into **eight 6-bit blocks**.

Each block is fed into a specific S-Box to substitute 6 bits with 4 bits.



This reduces the data back to **32 bits** while providing non-linear transformation.

S-Box Output (Binary):

0101 1100 1000 0010 1011 0101 1001 0111

Permutation (P-Box)

The 32-bit S-Box output is **rearranged using the P-box**.

This provides diffusion of the substituted bits across the output.



The P-box spreads the output bits from each S-Box across the entire 32-bit result.

P(S) Output (Binary):

0010 0011 0100 1010 1010 1001 1011 1011

Substitution Box 1 (S1)

■ S-Box Overview

- S-Boxes are **non-linear substitution** tables that transform 6-bit inputs into 4-bit outputs.
- DES uses **eight S-Boxes** (S1 through S8), each with a different substitution table.
- S-Boxes provide the **confusion** property in the Feistel network.

⚙️ How S1 Works

- The **row** is determined by bits 1 and 6 of the 6-bit input.
- The **column** is determined by bits 2 through 5.

Substitution Box 1 (S1)

Rows are determined by bits 1 and 6; Columns by bits 2-5.

Row	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

Permutation Function (P)

✂ P-Box Overview

- The Permutation Function (P) is a **32-bit table** that rearranges the output of the S-Boxes.
- Each entry specifies which input bit position moves to that output position.
- The P-box provides **diffusion** by spreading the S-Box outputs across the entire 32-bit result.

💡 Purpose

- The P-box ensures that the output bits from **each S-Box** are distributed across the entire 32-bit output.

P

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

Step 3: Round 1 Detail (E)



Final XOR and Swap

XOR the P-Box output with L_0 to produce R_1 .

The old R_0 simply becomes L_1 .



This implements the **Feistel structure**: the right half becomes the new left, and the transformed right becomes the new right.

→ R_1 Result

Binary:

1110 1111 0100 1010 0110 0101 0100 0100

Hexadecimal:

EF4A6544

← L_1 Result

Value:

F0AAF0AA

Note: L_1 is carried over directly from R_0 .

Step 4: End of Round 16

↔ After 16 Rounds

After completing the **16 rounds** of the Feistel network, we have calculated L_{16} and R_{16} .

i In the **very last round**, the standard block swap is **undone** (or skipped).

◀ L_{16}

Hexadecimal:

43423234

Binary: 0100 0011 0100 0010 0011 0010 0011 0100

▶ R_{16}

Hexadecimal:

0A4CD995

Binary: 0000 1010 0100 1100 1101 1001 1001 0101

↔ Pre-output

Structure ($R_{16} L_{16}$):

0A4CD99543423234

Note: Block is R_{16} followed by L_{16}
(swap undone)

Step 5: Final Permutation (IP^{-1})

↔ Process

The pre-output block is passed through the **Inverse Initial Permutation (IP^{-1})** table.



This permutation reverses the Initial Permutation (IP) that was applied at the start of encryption.



Purpose

- The final permutation restores the original bit order to produce the **final ciphertext**.
- This step ensures that the output bits are in the correct order as specified by the DES algorithm.



Final Ciphertext

Hexadecimal Value:

85E813540F0AB405



This is the **encrypted output** after completing all steps of the DES algorithm.



Binary: 1000 0101 1110 1000 0001 0011 0101 0100
0000 1111 0000 1010 1011 0100 0000 0101

Inverse Initial Permutation (IP^{-1})

↔ IP^{-1} Overview

- The Inverse Initial Permutation (IP^{-1}) is the **exact inverse** of the Initial Permutation (IP) table.
- It is a **64-bit table** that reverses the rearrangement performed by the IP table.
- This permutation is the **final step** in both encryption and decryption processes.

💡 Purpose

- IP^{-1} **restores the original bit order** to produce the final ciphertext (encryption) or plaintext (decryption).
- It ensures that the output bits are in the **correct order** as specified by the DES algorithm.

Inverse Initial Permutation (IP^{-1})

IP^{-1}

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

2



Decryption Process

Reversing the DES Algorithm



Same Algorithm



Reverse Keys



K_{16} to K_1

Decryption Process - Setup

Ciphertext

Hexadecimal Value:

85E813540F0AB405

 This is the **final output** from the encryption process.

Decryption Key

Hexadecimal Value:

133457799BBCDFF1

 Same key used during **encryption**.

Decryption Overview

- DES decryption uses the **exact same algorithm** as encryption.
- The key difference is that the **16 subkeys are applied in reverse order**.
- Round 1 of decryption uses K_{16} , Round 2 uses K_{15} , and so on down to K_1 .

Key Schedule

Decryption Round Order:

$K_{16} \rightarrow K_{15} \rightarrow \dots \rightarrow K_2 \rightarrow K_1$

Decryption Process - Step 1: Initial Permutation (IP)

↔ Process

The ciphertext undergoes the **standard Initial Permutation (IP)**.



Because the ciphertext was created by passing through **IP^{-1}** at the end of encryption, this step **perfectly neutralizes** the final permutation of the encryption phase.

➡ IP Output

Hexadecimal Value:

0A4CD99543423234

Notice this is exactly **R_{16}** **L_{16}** from encryption.

↗ Split into Halves

The IP output is split into Left and Right halves.

L_0' (was R_{16}):

0A4CD995

R_0' (was L_{16}):

43423234



Key Insight

Since they enter the network reversed from how they left, we label them as **L_0'** and **R_0'** .

Decryption Process - Step 2: Key Generation (Decryption Round 1)

Key Generation Process

- We generate the keys via the **same schedule** as encryption.
- However, we grab the **final key**, K_{16} , to use first in decryption.

Key Schedule

Decryption Round Order:

Round 1: K_{16}
Round 2: K_{15}
Round 3: K_{14}
...
Round 16: K_1

Round 1 Subkey (K_{16})

Binary Representation:

110010 110011 110110 001011
000011 100001 011111 110101

i This is the **same K_{16}** that was used in the **last encryption round**.

Reverse Application

By applying the keys in **reverse order**, the Feistel network **mathematically unwinds** itself layer by layer, perfectly reversing the encryption process.

Decryption Process - Step 3: Round 1 Detail

⚙️ Round Function

$$f(R_0', K_{16})$$

We apply the Feistel function using the **reversed inputs** and **reversed keys**: $f(R_0', K_{16})$.

● Expansion (E-Box)

R_0' (43423234) is expanded to 48 bits.

Σ Mathematical Property

$$f(L_{16}, K_{16}) = \text{same output}$$

Because of the mathematical property of the **Feistel structure**, the function $f(L_{16}, K_{16})$ yields the **exact same bit pattern** emerging from the P-box during decryption as it did in encryption round 16.

Decryption Process - Step 3: Round 1 Detail

Key Mixing (XOR)

The expanded R_0' is XORed with K_{16} .

Substitution & Permutation

The result passes through S-Boxes and P-box.

Final XOR and Swap

- XOR the P-box output with L_0' (which holds the value of R_{16}).

Result:

$$R_1' = L_0' \text{ XOR } f(R_0', K_{16})$$

- This **perfectly reverses** encryption round 16, giving us the L_{15} and R_{15} states from the encryption phase.
- L_1' becomes the old R_0' (43423234).

Decryption Process - Step 4: End of Round 16

🏆 Final Decryption Round

- In the final decryption round, we use the **very first subkey, K_1** .
- After the 16th round, the block swap is **skipped again**, leaving the block as $R_{16}' L_{16}'$.

🌟 Mathematical Unwinding

Because the entire mathematical process has been **unwound**, this block is identical to the data state from **just after** the Initial Permutation during Step 1 of encryption.

↩️ Decryption Summary

- The Feistel network **mathematically unwinds** itself layer by layer.
- Each decryption round perfectly reverses the corresponding encryption round.
- The pre-output block is now ready for the **final permutation**.

Decryption Process - Step 4: End of Round 16

Pre-output Block

CC00CCFFF0AAF0AA

This matches the **IP output** from encryption Step 1.

Key Insight

The **reverse key application** ($K_{16} \rightarrow K_1$) ensures that each decryption round exactly cancels out its corresponding encryption round, restoring the original plaintext.

Decryption Process - Step 5: Final Permutation (IP^{-1})

↔ Final Permutation

- The pre-output block is passed through the **Inverse Initial Permutation (IP^{-1})** table.
- This **cleanly reverses** the very first step of encryption.

Σ IP^{-1} Function

The IP^{-1} table is the **exact inverse** of the Initial Permutation (IP) table used at the beginning of the encryption process.

✓ Plaintext Output

Hexadecimal Value:

0123456789ABCDEF



Original Plaintext Recovered!

✓ Verification

- The output **exactly matches** the original plaintext from the encryption setup.
- This confirms that the **decryption process** successfully reverses the encryption.



Acknowledgement

The material presented in these slides incorporates content from books, academic lecture notes, online resources, and generated text. All rights remain with the original authors and publishers. This content is used solely for educational purposes to illustrate and clarify concepts. No copyright infringement is intended.